Gegevensstructuren en Algoritmen
Opgave project 2024

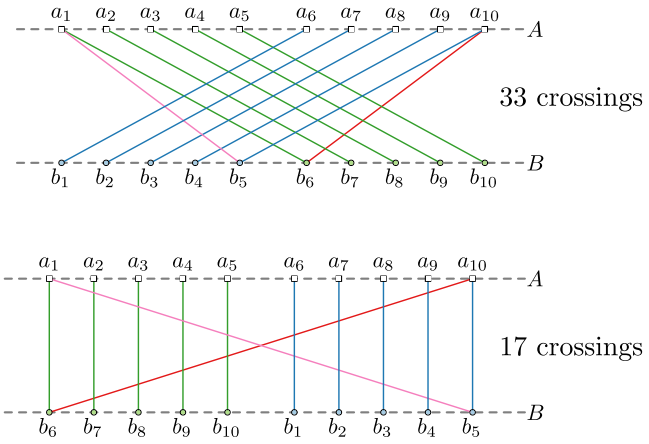
Jarne Renders, Jan Goedgebeur
jarne.renders@kuleuven.be, jan.goedgebeur@kuleuven.be

1 Project

In dit project starten we vanuit volgende paper:

Rafael Martí and Manuel Laguna. Heuristics and meta-heuristics for 2-layer straight line crossing minimization. *Discrete Applied Mathematics*, 127(3):665–678, 2003.

In deze paper bestuderen Martí en Laguna verschillende methodes om het bipartite drawing problem (BPD) op te lossen. Dat probleem is gedefiniëerd als volgt. Gegeven is een bipartiete graaf, i.e. een graaf waarvan we de knopen kunnen verdelen in twee sets A en B zodat knopen in A niet verbonden zijn met knopen in A en knopen in B niet verbonden zijn met knopen in B. We kunnen de knopen van A in een bepaalde volgorde op één lijn tekenen en hiertegenover de knopen in B in een bepaalde volgorde op één lijn tekenen. Als we de bogen van de graaf met rechte lijnen tekenen, hebben we dan een vast aantal snijpunten (crossings) tussen verschillende bogen. Zie Figuur 1 als voorbeeld. Het BDP vraagt het minimum aantal crossings dat je kan bekomen door de volgorde van A en B te wijzigen.



Figuur 1: Twee volgordes van de knopen in B voor de gegeven graaf.

Het BDP is NP-compleet en dus is het niet mogelijk om in redelijke tijd voor grotere instanties het minimale aantal crossings te bepalen. Hierdoor wordt in de literatuur vaak gekeken naar heuristieken en metaheuristieken omdat deze in korte tijd tot een vrij goede (maar wellicht niet optimale) oplossing kunnen komen.

Het artikel beschrijft verschillende heuristische methoden om het probleem aan te pakken en twee metaheuristieken. Ze testen al deze algoritmes op een heleboel benchmarkgrafen en vergelijken ze onderling.

De structuur van het artikel is als volgt:

(1) Introductie/probleembeschrijving,

(2) Uitleg over de heuristieken,

(3) Uitleg eerste metaheuristiek,

(4) Uitleg tweede metaheuristiek,

(5) Experimenten en resultaten.

We verwachten van jullie dat jullie:

- De paper lezen en globaal begrijpen. (Je hoeft niet alles vanbuiten te kennen, maar enkel in grote lijnen te weten waar elke sectie over gaat.)
- Opzoeken wat Local Search en Tabu Search is.
- Twee heuristieken uit de paper en een metaheuristiek kiezen om nu te begrijpen en later te implementeren:
  - Voor een introductie, zie sectie 2.1 van `http://www.scholarpedia.org/article/Metaheuristics`.
  - Kies uit: Simulated Annealing (SA), Tabu Search (TS) of Iterated Local Search (ILS).
- De twee heuristieken en gekozen metaheuristiek implementeren in Java om het BDP op te lossen.
- Experimenten uitvoeren en de resultaten rapporteren in een verslag van ongeveer 8 pagina's.
- Het project mondeling verdedigen.

2 Planning

De planning en verwachtingen zijn als volgt:

2.1 Begrijpen

In de eerste fase is het begrijpen van de paper en de (meta)heuristieken belangrijk. De paper gaat niet in zeer veel detail in op hoe de (meta)heuristieken werken. Kijk daarom eens tussen de referenties om de heuristieken beter te begrijpen en zoek eens op hoe Tabu Search werkt. Belangrijk is echter dat jullie goed weten wat het probleem inhoudt, in grote lijnen weten hoe de algoritmes van de auteurs werken en in detail weten hoe de gekozen heuristieken en metaheuristiek werkt.

Op **donderdag, 31/10,** 13:00-15:00, zal de eerste sessie plaatsvinden. Jullie kunnen hier al wat aan het project werken. Tijdens deze sessie zal elke student individueel komen uitleggen aan de assistent waar het artikel over gaat en hoe de gekozen heuristieken/metaheuristiek werkt. Het doel hiervan is om ervoor te zorgen dat jullie de paper en het gekozen algoritme goed begrijpen alvorens te beginnen aan het ontwerp en de implementatie, zo niet kunnen we jullie vroeg bijsturen. We verwachten dus dat jullie goed voorbereid zijn om het artikel en de gekozen metaheuristiek uit te leggen. Aarzel zeker niet om tijdens dit moment vragen stellen over zaken die niet (helemaal) begrepen zijn.

2.2 Analyse en ontwerp

In deze stap moeten de studenten hun gekozen heuristieken en metaheuristiek analyseren en een datastructuur kiezen/ontwikkelen voor de implementatie. Deze stap is zeer belangrijk, want de gekozen datastructuur is een van de belangrijkste factoren voor de efficiëntie en flexibiliteit van de implementatie. Denk na over de oplossingsruimte van je algoritme, of je een alternatieve objectieffunctie zou kunnen nemen en hoe je van de ene naar de andere oplossing kan gaan.

De tweede meeting vindt plaats op **donderdag, 14/11,** 13:00-15:00. Hier kunnen jullie individueel uitleggen welke datastructuren er gekozen zijn en het ontwerp (kort) uitleggen aan de hand van een flowchart/UML diagram of pseudocode. De assistent zal tijdens deze sessie ook feedback geven op het ontwerp zodat hier indien nodig nog bijgestuurd kan worden.

2.3 Implementatie

De volgende meetings zullen gaan over de implementatie in Java van de algoritmes en het uitvoeren van eerste experimenten. Op **maandag, 25/11,** 9:30-11:00 en 16:00-17:30, zullen de studenten individueel uitleg moeten geven over hun eerste implementatie, met hier ook de kans om vragen te stellen.

Op **maandag, 02/12,** 9:30-11:00, zullen de studenten een tweede versie van de implementatie met de assistent kunnen bespreken, gebruikmakend van de feedback van de vorige sessie en al enkele eerste resultaten tonen.

2.4 Verslag en verdediging

In de laatste fase moeten de studenten een verslag schrijven waarin ze het project samenvatten, hun ontwerpsbeslissingen toelichten (o.a. keuze algoritmes en datastructuren), hun implementatie analyseren en hun experimenten beschrijven en de resultaten bespreken. We verwachten dat dit verslag ongeveer acht pagina's lang is.

Probeer interessante onderzoeksvragen te stellen. Enkele voorbeelden zijn:

- Is de metaheuristiek efficiënter dan enkel de simpele heuristieken?
- Wat is het effect van verschillende startoplossingen?

- Hoe verhouden jouw resultaten zich in vergelijking met die uit de paper?

Ten laatste tegen **zondag, 08/12,** 23:59, moeten de studenten via Toledo een zip file indienen met daarin:

- hun broncode samen met een README bestand waarin uitgelegd staat hoe de code te compileren en hoe de output geïnterpreteerd moet worden.
- het finaal verslag als pdf.

De studenten moeten op **donderdag, 12/12,** individueel hun project komen verdedigen. Hierbij bespreken ze kort hun verslag en moet de student hierover een aantal vragen beantwoorden.

3 Evaluatie

Dit project telt mee voor 7 van de 20 punten voor dit vak. Jullie worden beoordeeld op basis van de volgende criteria:

- Kwaliteit van de code
  - leesbaarheid,
  - commentaar,
  - uitbreidbaarheid,
  - volledigheid en correctheid.
- Kwaliteit van het verslag
  - duidelijk geschreven,
  - niveau van detail,
  - beschrijving van de experimenten en resultaten.
- Prestatie algoritme t.o.v. benchmarks
- Experimenten
  - aantal,
  - relevantie.
- Mondelinge verdediging

4 Algemene richtlijnen

- Jullie moeten dit project maken in Java. Er wordt verwacht dat jullie code compileert met Java 17.
- Extra externe libraries zijn niet toegestaan behalve de Java Development Kit (JDK) en JUnit.

- Jullie zullen zelf een klasse `BipartiteGraph.java` moeten schrijven die de interface `BipartiteGraphInterface.java` implementeert (te vinden op Toledo). Uiteraard mogen jullie daarnaast zoveel klassen aanmaken als jullie willen.
- Als IDE raden we jullie IntelliJ IDEA Ultimate aan, maar hier hebben jullie de vrije keuze.
- Dit project is individueel en dient dus door jou persoonlijk gemaakt te worden. Het is niet toegestaan om code uit te wisselen of over te nemen van het internet. We gebruiken geavanceerde plagiaatdetectiesoftware om ongewenste samenwerking te detecteren. Zowel het gebruik van andermans werk, als het delen van werk met anderen, zal als examenfraude worden beschouwd en zal conform het examenreglement behandeld worden.
- Voor regelgeving rond het gebruik van generatieve AI verwijzen we naar: `https://www.kuleuven.be/onderwijs/student/onderwijstools/artificiele-intelligentie`. Het is prima om dit te gebruiken voor het herwerken van kleine stukjes tekst of het schrijven van triviale stukjes code (stel jezelf wel altijd de vraag of je dit zelf ook had kunnen schrijven en controleer de correctheid!), maar elk gebruik moet duidelijk vermeld worden.
- Op Toledo vinden jullie een heleboel grafen voor benchmarking. Hieronder zitten 900+900+1000 grafen gegenereerd met GraphBase, die jullie kunnen gebruiken om de experimenten van Martí en Laguna zelf uit te voeren. Daarnaast vinden jullie nog 13 hele kleine grafen om je code op te testen, 200 relatief eenvoudige instanties en 200 moeilijke instanties.
- Een uitleg over het formaat van deze grafen is ook te vinden op Toledo.